ESP32-Based Drone Development Internship – Technical Overview

During my internship at AIMP Labs, I was involved in the design and development of an ESP32-based quadcopter with the objective of understanding the complete embedded hardware development cycle. The project involved integrating electronic components, developing embedded firmware, designing the power stage, establishing wireless communication, and studying the control mechanisms required for stable flight. Rather than focusing on a single subsystem, the internship provided exposure to how multiple hardware and software modules interact to form a complete autonomous embedded system.

The ESP32 microcontroller served as the central processing unit of the drone due to its dual-core architecture, integrated Wi-Fi and Bluetooth capabilities, multiple PWM channels, and sufficient processing power for real-time embedded applications. The firmware was responsible for initializing peripherals, generating PWM signals, handling wireless communication, processing user commands, and coordinating the operation of the entire system. Understanding the timing constraints of embedded systems and ensuring deterministic execution of tasks was an important part of the development process.

The propulsion system consisted of four Brushless DC (BLDC) motors, each controlled by an Electronic Speed Controller (ESC). Since BLDC motors cannot be driven directly from a microcontroller, the ESP32 generated PWM signals that acted as throttle commands for the ESCs. Each ESC interpreted the PWM duty cycle and internally performed three-phase electronic commutation using high-speed MOSFET switching to regulate the speed and torque of its respective motor. Learning the interaction between the microcontroller, ESCs, and BLDC motors provided valuable insight into modern motor control systems.

A significant part of the internship involved understanding the power electronics required for reliable motor operation. N-channel MOSFETs were used as high-speed electronic switches because of their low on-state resistance and fast switching characteristics. Considerable attention was given to proper gate driving techniques, current handling capability, power distribution, grounding, and protection against voltage transients generated by inductive loads. These concepts highlighted the importance of efficient power management and circuit reliability in embedded hardware design.

Wireless communication was implemented using the ESP32's built-in Wi-Fi and Bluetooth modules, allowing a mobile application to remotely control the drone. Control commands such as throttle, roll, pitch, and yaw were transmitted wirelessly, received by the ESP32, decoded into usable control values, and converted into PWM outputs for the ESCs. This required developing a reliable communication interface while minimizing latency and ensuring smooth real-time response between the user and the drone.

To better understand how modern drones maintain stable flight, I studied the integration of the MPU6050 Inertial Measurement Unit (IMU), which combines a three-axis accelerometer and a three-axis gyroscope. The sensor continuously measures the drone's angular velocity and linear acceleration, allowing the flight controller to estimate its orientation in terms of roll, pitch, and yaw. Since raw accelerometer data is susceptible to vibration and gyroscope data accumulates drift over time, sensor fusion techniques such as complementary filtering are commonly used to obtain accurate and stable orientation estimates.

Based on the orientation data obtained from the IMU, PID (Proportional–Integral–Derivative) control algorithms are typically employed to stabilize the drone during flight. The controller continuously compares the desired orientation with the measured orientation to calculate an error value. The proportional term

provides an immediate corrective response, the integral term compensates for accumulated steady-state error, and the derivative term predicts future error by considering the rate of change, thereby reducing oscillations. The output of the PID controller is then translated into motor speed adjustments, allowing the drone to maintain balance and quickly recover from external disturbances such as wind or sudden movements.

Although the primary focus of the internship was hardware development and wireless control, studying the principles of IMU-based stabilization and closed-loop PID control provided a deeper understanding of complete flight controller architecture. It demonstrated how sensors, controllers, actuators, communication systems, and embedded software operate together within a real-time control system to achieve stable and reliable flight.

Throughout the project, hardware debugging formed an essential part of the development process. Individual subsystems were verified independently by checking PWM signal generation, calibrating ESCs, confirming motor rotation direction, validating wireless communication, monitoring power supply stability, and troubleshooting wiring issues. This systematic approach to testing emphasized the importance of modular verification, fault isolation, and iterative debugging when developing complex embedded systems.

Overall, the internship provided practical exposure to embedded systems design, microcontroller programming, motor control, power electronics, wireless communication, sensor integration, real-time control concepts, and hardware-software co-design. Beyond developing technical skills, it reinforced the importance of structured problem-solving, documentation, systematic testing, and understanding how individual engineering components integrate into a complete embedded system.